CompoVisor: Authoring Composite Visualization From Chart Blocks

Category: Research

Figure 1: Dramatic evening clouds. Note that the teaser may not be wider than the abstract block.

Abstract—Composite visualizations combine multiple charts through operations such as concatenation, layering, and faceting to reveal relationships across views. However, existing authoring tools often require users to manipulate low-level marks or predefined specifications, making it difficult to compose, adapt, and iteratively refine visualizations at the chart level. We present CompoVisor, an interactive authoring tool that treats charts as reusable and editable blocks for constructing composite visualizations. To inform the definition and organization of these blocks, we conducted a formative study with visualization experts and identified three key dimensions—coordinate system, visual style, and data dimensionality—for characterizing chart blocks and their alternatives. In CompoVisor, users construct visualizations by dragging and dropping chart blocks to express composition relationships, which remain explicit and editable throughout the authoring process. A data-aware recommendation engine evaluates dimensional compatibility among blocks and suggests alternative chart designs and data mappings to resolve mismatches and support design exploration. Through case studies and a comparative user study, we demonstrate that CompoVisor supports the construction and iterative refinement of complex composite visualizations and is easier to learn and use than Lyra. Our work demonstrates how block-based interaction and data-aware recommendations can support accessible and flexible composite visualization authoring.

Index Terms—Visualization Authoring Tool, Composite Visualization, Drag and Drop

1 Introduction

Many data-driven graphics combine multiple visual representations to compare data, reveal cross-view relationships, and communicate complex narratives. Such composite visualizations are common in visual analytics and communication [7, 10, 28, 29], encompassing patterns such as concatenation, layering, faceting, and nesting [6, 8, 31]. Authoring these visualizations requires managing data, encodings, coordinate systems, layouts, and dependencies across charts.

Existing approaches address different stages of visualization authoring, but chart-level composition often remains indirect. Low-level libraries [3] provide fine-grained control, but require authors to manually implement and maintain alignment, shared scales, layouts, and update logic. Interactive graphical editors similarly offer substantial freedom in constructing and manipulating marks and visual primitives [12, 21]. While this mark-level flexibility supports expressive designs, authors must first assemble primitives into coherent charts and then maintain their relationships during composition and subsequent editing. Declarative grammars and template-based tools facilitate common charts and compositions, but require authors to work with predefined operators, specification structures, or layout templates [1, 3, 22]. Departures from these structures may require substantial specification changes or low-level customization. These approaches leave an opportunity for authoring abstractions that operate directly on charts while retaining interactive control over their composition and refinement.

Realizing this opportunity raises three challenges. First, composing charts as blocks involves more than arranging them spatially: adjacent charts may share an axis, overlapping charts may form a semantic layer, and repeated charts may constitute a facet. These composition relationships must remain explicit and editable so that individual blocks and their dependencies are preserved during refinement. Second, data adaptation becomes more complex once charts are composed. For an individual chart, incompatibilities can often be addressed by remapping data fields and encodings. A composition, however, introduces cross-chart constraints: its blocks may differ in data dimensions, schemas, or levels of granularity, while shared axes, layers, and facets impose different compatibility requirements. The authoring process must therefore help authors identify and reconcile such mismatches without reconstructing the constituent charts. Third, the predefined scope of chart blocks may constrain design exploration. A block-based approach should therefore organize blocks along meaningful design dimensions and surface alternatives for authors to explore. Although blocks may serve as reusable templates, they should remain configurable and replaceable rather than prescribe fixed visual outcomes.

To inform the design of chart blocks and their alternatives, we first conducted a formative study with visualization experts. Drawing on perceptual principles and their design experience, the experts examined what should constitute a reusable chart block and which alternatives should be offered for a given design. Our analysis of their discussions identified three dimensions for organizing chart blocks—coordinate system, visual style, and data dimensionality. These dimensions provide a structured basis for defining block units and surfacing design

alternatives during authoring.

Guided by these findings, we developed CompoVisor, an interactive authoring tool for constructing composite visualizations with reusable chart blocks. Users drag and drop chart blocks to express spatial relationships, which the system interprets as composition operations such as concatenation, layering, and faceting. A data-aware recommendation engine evaluates the dimensional compatibility of the blocks and recommends alternative chart designs and data mappings, both to resolve mismatches and to support design exploration. These alternatives are organized along the coordinate-system, visual-style, and data-dimensionality dimensions identified in our formative study. Users can then rearrange, replace, and refine individual blocks while their composition relationships remain explicit and editable. Through case studies and a comparative user study, we demonstrate that CompoVisor supports the construction and iterative refinement of complex composite visualizations and is easier to learn and use than Lyra [21].

In summary, our contributions are:

- A formative study that identifies coordinate system, visual style, and data dimensionality as key dimensions for defining reusable chart blocks and organizing their design alternatives;
- An interactive authoring tool that supports drag-and-drop composition with explicit and editable chart relationships, together with a data-aware recommendation engine for evaluating compatibility and suggesting chart alternatives and data mappings;
- Empirical findings from case studies and a comparative user study that characterize the capabilities, benefits, and limitations of block-based, data-aware authoring for composite visualizations.

2 Related Work

We review prior work in design patterns, libraries and grammars, and interactive design tools for composite visualization.

2.1 Design Patterns of Composite Visualization

Composite visualizations integrate multiple visual representations to support complex analytical and communicative goals and are widely used in visual analytics systems [7, 10, 28, 29]. Compared with single-view visualizations, their design requires consideration of not only the encodings within individual charts but also the spatial, semantic, and data relationships among charts [6, 13, 17, 31].

Prior work has characterized composite visualizations through a range of composition patterns. Early taxonomies identify high-level patterns such as juxtaposition, superimposition, overloading, and nesting [8]. Subsequent work provides more fine-grained characterizations of their visual variations [6], while recent schema-based representations formalize composite designs using operators such as concatenation, layering, faceting, and nesting and capture the data types encoded by their components [31]. Together, these studies establish a useful vocabulary for describing composite visualization structures.

However, these representations are primarily designed to characterize existing visualizations rather than support their construction. Interactive authoring additionally requires explicit decisions about the granularity of reusable components and the conditions under which they can be composed. For example, visually related chart variants may require different data dimensions, while composition operations may impose constraints on shared fields, coordinate systems, or data hierarchies. Leaving these requirements abstract makes descriptive patterns difficult to translate directly into interactive authoring operations.

CompoVisor builds on these taxonomies by operationalizing their composition patterns for authoring. It defines reusable chart-level building blocks together with their data-dimensional requirements and enables users to compose them through direct manipulation.

2.2 Libraries and Grammars for Composite Visualization

Visualization libraries [2, 3, 11, 20, 27] and grammars [4, 9, 11, 14, 16, 18, 22–25, 32, 34] provide formal mechanisms for authoring visualizations.

In composite visualization authoring, low-level libraries such as D3 [3] offer fine-grained control over different visual marks, but require authors to manually implement relationships among them, including alignment, layout coordination, and shared scales. Higher-level declarative grammars such as Vega-Lite [22] reduce some of this implementation burden by providing operators and configuration options for common cases such as concatenation, faceting, and layering. Nevertheless, these compositions are still specified through textual or hierarchical structures, rather than through direct associations among visual objects. When compositions become complex, authors often need high familiarity with the specification structure and produce lengthy hierarchical descriptions. Hence, expressing visual composition intents through code or declarative specifications remains indirect for iterative canvas-based arrangement, nesting, layering, and refinement.

CompoVisor adopts a complementary approach by maintaining an internal schema for composite visualization authoring. The schema represents composition relationships among visual components and is incrementally updated through direct manipulation. In this way, users can construct complex composite structures through direct manipulation, without manually adjust complex specifications.

2.3 Interactive Visualization Design Tools

Numerous interactive tools [1, 5, 12, 19, 21, 30, 33] have been developed to support visualization authoring without conventional programming.

Chart typology tools and online chart makers [5, 15, 19, 26] allow users to quickly create common charts from data and compare design alternatives. More advanced systems such as Tableau [1] provide richer interactive authoring workflows, including shelf-based encoding, automatic chart generation, and detailed configuration of visual mappings. Nevertheless, these tools are primarily oriented toward single-view visualizations or dashboard-style arrangements and provide limited support for constructing composite visualizations through explicit relationships among their constituent charts, such as layering, faceting, and nesting.

Research systems further explore how direct manipulation and graphical editing can support more expressive visualization design. Lyra [21] combines data-driven visualization specification with familiar vector-graphics-style editing, while Data Illustrator [12] introduces lazy data binding to flexibly transform graphical elements into data-driven graphics. However, these systems primarily construct visualizations from low-level graphical elements, such as rectangles, lines, and arcs, which enables highly customized designs but may require users to construct and coordinate visual structures from relatively primitive building blocks, even when the intended design consists of recognizable charts.

In contrast, CompoVisor treats charts as the primary building blocks and supports direct manipulation of chart-level composition relationships, reducing the need to construct composite visualizations from low-level graphical primitives.

3 Authoring-Oriented Design Space

We translate existing descriptive abstractions of composite visualizations into an authoring-oriented design space. This process combines two sources of insight. First, we conducted a study with visualization experts to establish the granularity and perceptual organization of composable chart blocks. Second, building on prior composition models, we operationalized composition in terms of data and spatial relationships and examined how these relationships are realized across coordinate systems. Together, these analyses define the units, relationships, and spatial structures that form the conceptual basis of CompoVisor.

3.1 From Descriptive to Authoring-Oriented

Prior work [6, 8, 31] provides descriptive abstractions for characterizing existing composite visualizations, including composition types such as layering, concatenation, faceting, and nesting [31]. Authoring, however, requires operationalizing these abstractions to support the construction of new compositions. This shift raises three questions that are largely implicit in existing design spaces: what units should be composed, what data relationships enable their composition, and how composition is realized across coordinate systems.

Chart blocks. An authoring system requires concrete and manageable units of composition. Existing vocabularies commonly describe views using coarse chart types, but such types may encompass structurally distinct idioms. For example, grouped, stacked, normalized stacked, and diverging stacked bar charts differ in their visual structures and data requirements. Conversely, treating every minor visual variation as a separate unit would unnecessarily fragment the design space. An authoring-oriented design space must therefore establish a block granularity that captures consequential structural differences while abstracting away stylistic variations.

Data dimensions. Chart idioms require different combinations of categorical, quantitative, temporal, geographic, hierarchical, or relational dimensions. Composition further introduces relationships among the data used by different blocks, such as shared positional dimensions for layering, partitioning dimensions for faceting, or correspondence keys for nesting. These data requirements and cross-block relationships must be made explicit to guide how blocks can be composed.

Coordinate systems. Existing composition models are largely grounded in two-dimensional Cartesian space. Authoring across other coordinate systems requires these relationships to be interpreted through different spatial dimensions. For example, polar coordinates support angular and radial arrangements, whereas geographic coordinates introduce projection- and reference-dependent placement. An authoring-oriented design space must therefore distinguish abstract composition types from their coordinate-specific spatial realizations.

Together, these considerations extend existing descriptive abstractions into an authoring-oriented design space organized around composable chart blocks, their data relationships, and their spatial realization across coordinate systems.

3.2 Expert Study of Composable Chart Blocks

Chart blocks define the units that an authoring system can instantiate, manipulate, and compose. We establish their granularity and organization through an expert-grounded analysis of the VAID corpus [31], which contains 442 composite visualizations.

Expert involvement. We recruited 6 visualization experts (E1–E6), including 4 academic researchers (E1–E4) and 2 industry practitioners (E5, E6). All experts had more than 5 years of professional or research experience in visualization and had designed and implemented composite visualizations in the development of visual analytics systems or related applications. In addition, E1 and E3 had prior experience conducting visualization surveys and had systematically studied the classification, application contexts, and visual forms of visualizations. Their complementary expertise in visualization research, system development, and practical design informed both the decomposition of composite visualizations into chart blocks and the subsequent organization of these blocks into perceptually meaningful families.

Block granularity. The experts independently decomposed 100 representative visualizations sampled from the corpus into constituent chart units. Without a predefined taxonomy, they identified units that they considered visually coherent and meaningful to construct or manipulate independently. We compared their decompositions to identify recurring block boundaries, obtaining substantial inter-rater agreement (Fleiss' $\kappa = 0.74$). Based on these judgments, we distinguish blocks when their structural differences affect data requirements, encoding behavior, or subsequent composition. For example, grouped, stacked, normalized stacked, and diverging stacked bar charts are treated as distinct blocks because they organize marks and data differently, whereas minor stylistic variations do not generally define separate blocks.

Perceptual organization into families. The experts then organized the resulting blocks into chart families through an association-driven grouping task. Rather than asking them to derive a formal taxonomy, we asked which blocks they would expect to find under the same broad category in an authoring tool and which ones they would regard as design alternatives. We converted their co-grouping judgments into pairwise similarities and used hierarchical clustering and majority agreement to derive candidate families, while retaining their qualitative rationales to interpret relationships within each family. This procedure prioritizes how authors recognize and search for chart designs rather than how developers might organize them based solely on implementation similarity. The experts' rationales revealed three recurring factors that shape family organization and the presentation of alternatives:

Coordinate system. A change of coordinate system can produce chart idioms that are structurally related but perceptually distinct. For example, icicle plots and sunburst charts represent similar hierarchical structures in Cartesian and polar coordinates, respectively. Most experts initially identified them as distinct chart types, whereas E1 and E3, who had prior experience developing chart classifications, explicitly associated them as coordinate-system alternatives. This suggests that an authoring tool should preserve both idioms as recognizable options while making their structural relationship accessible.

Data dimensionality. Differences in data requirements frequently define alternatives within a family. This consideration was especially prominent for bar charts: experts associated grouped and stacked variants with the basic bar chart while recognizing that they introduce additional data dimensions and corresponding mark organizations. Such variants should therefore remain connected within a family without being collapsed into a single block.

Visual style. Stylistic differences can also affect how users identify and retrieve a chart. E1 and E3 regarded a step line chart primarily as a variant of a line chart. E4 agreed with this relationship but emphasized that step lines are often sought as a distinct, domain-specific idiom, for example in financial visualization, an author with such a use case may search directly for a step line chart rather than derive it as a style variation of a line chart. We therefore use family relationships to connect stylistic alternatives while retaining directly recognizable blocks where appropriate.

Block specification. Each block is specified by its chart idiom, data requirements, encoding channels, and supported coordinate systems. Drawing on the visualization primitives and encoding abstractions of D3.js [3] and Vega-Lite [22], we annotate the visual channels supported by each block, including position, length, color, size, shape, and angle. The chart idiom describes the block's visual structure, such as a grouped bar chart, line chart, scatterplot, or area chart. Its data requirements specify the number and semantic roles of the required dimensions, with each dimension annotated by data type, such as categorical, quantitative, temporal, geographic, hierarchical, or relational. Coordinate support identifies the spatial dimensions and coordinate systems through which these data dimensions are visually encoded.

3.3 Dimension-Aware Composition

Existing composition types have been derived primarily from visualizations in two-dimensional Cartesian space. We therefore first operationalize these types in terms of their relationships along the x and y dimensions. In particular, we require a composition to couple blocks in both data space and display space: the blocks must have an operator-specific relationship between their data dimensions or entities, and this relationship must be reflected by their spatial arrangement. Blocks that are merely aligned or placed adjacent without a data-space relationship are treated as layout arrangements, whereas blocks that use related data but are positioned independently remain separate charts. Following VAID [31], we consider four composition types: layering, concatenation, faceting, and nesting.

Layering places multiple blocks in the same plotting region and couples them along both coordinate dimensions. In Cartesian space, layered blocks share both the x and y axes, including compatible dimensions, scales, and spatial positions. For example, a line chart and an area chart can be layered when their horizontal and vertical encodings can be expressed in a common coordinate space.

Concatenation places blocks in adjacent plotting regions while coupling them along one coordinate dimension. Horizontally concatenated blocks share and align their y axis while occupying separate regions along x; vertically concatenated blocks share and align their x axis while occupying separate regions along y. Thus, adjacency alone does not constitute concatenation: blocks placed side by side without a shared and spatially aligned axis are treated as a layout arrangement.

Faceting partitions data by one or more dimensions and instantiates a common chart block for each partition. The instances preserve the

same chart idiom and encoding structure and are systematically positioned according to the faceting dimensions. Faceting therefore couples instances through both a shared visualization specification and a spatial arrangement determined by the partitioning dimension.

Nesting associates child-block instances with a collection of repeated internal elements in a parent block. Nesting applies the same child-block specification to each eligible parent element. These elements may be data-driven marks, such as points in a scatterplot, or repeated structural components, such as axes in a parallel coordinates plot. For each parent element, the system instantiates a child block and establishes its data-space relationship through a shared key, a filtering condition, or a hierarchical correspondence. The child instance is then positioned relative to the spatial reference of the corresponding element, such as its position, extent, or boundary.

Under this definition, layering shares two coordinate dimensions, concatenation shares one, faceting derives repeated blocks from a partitioning dimension, and nesting establishes a parent–child correspondence. In each case, composition is distinguished from general layout by the joint presence of data-space and display-space relationships.

3.4 Extending Composition Across Coordinate Systems

The preceding definitions operationalize composition in Cartesian space. We further consider how these composition types extend to other coordinate systems represented on a two-dimensional plane.

Coordinate-system requirements. Layering and concatenation require the participating blocks to use the same coordinate system, since they share two and one of its spatial dimensions, respectively. Nesting does not impose this requirement: the parent and child may use different coordinate systems because their relationship is established through a parent anchor. Faceting is similarly independent of the coordinate systems of the repeated blocks, it introduces an outer coordinate system that determines how the block instances are arranged.

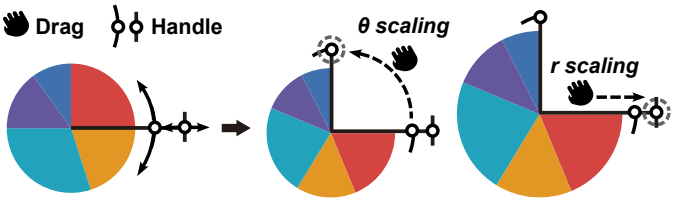

Figure 2: Interactive polar scale control. Our design enables independent scaling of angular (θ) and radial (r) dimensions. Left to right: origin scale; adjust angular range via θ handle; modify radial extent via r handle.

Polar coordinates. Polar coordinates provide an angular dimension θ and a radial dimension r. Of particular interest is concatenation, which has two coordinate-specific forms: angular concatenation partitions θ while sharing r, producing adjacent sectors, whereas radial concatenation partitions r while sharing θ, producing concentric rings. The same dimensions also define the arrangement of a polar facet. Notably, angular concatenation may assign different angular sectors to different chart blocks rather than requiring each block to span the full 0°–360° range. For example, a pie chart and a polar dot plot can each occupy one half of the polar space while sharing the radial dimension (Figure 3.B2). However, traditional polar visualizations assume a fixed full-circle range. To address this, we introduce interactive scaling controls (Figure 2) that enable dynamic adjustment of both θ range and r scale through direct manipulation handles on the axes, providing more flexible control over the polar coordinate space.

Geographic coordinates. Geographic coordinates use longitude and latitude to specify locations rather than allocate independent chart regions. We therefore support layering in a shared geographic reference and nesting through geographic anchors, such as points or regions. Concatenation and faceting are not defined directly over geographic coordinates because they require additional assumptions about projections, boundaries, or regional partitions.

Coordinate-free layouts. Coordinate-free layouts position elements either through layout algorithms, such as tree or force-directed layouts, or through positions explicitly specified by the user. Although these layouts assign positions to elements, the positions do not constitute shared spatial dimensions with explicit coordinate semantics. They therefore do not directly support the dimension sharing required by layering and concatenation. Coordinate-free blocks may nevertheless participate in nesting through element-level anchors or appear within a facet whose arrangement is defined by an external coordinate system.

4 Design Rationale

Based on the challenges identified in composite visualization authoring and the findings from authoring-oriented design space, we derived two design rationales for CompoVisor. First, spatial layout should provide a direct reference for expressing composition intent, while composition relationships remain explicit and editable. Second, composition-aware recommendations should help resolve incompatibilities and enable the exploration of alternatives even when blocks are already compatible.

4.1 Layout as a Reference for Composition

Spatial layout provides a natural reference for composing chart blocks, but spatial proximity alone does not necessarily imply a composition. An authoring tool should therefore distinguish free-form arrangement from composition and, once blocks are composed, preserve their established spatial relationships during subsequent editing.

Distinguish composition from free-form arrangement. Spatial interactions should allow users to indicate whether blocks are merely placed near one another or intended to form a composition. This distinction preserves the flexibility of free-form layout while enabling users to establish composition types through direct manipulation.

Preserve the internal layout of a composition. Once blocks are composed, their relative positions become part of the composition and should be maintained when it is manipulated in a broader layout. The composition should therefore behave as a coherent unit during external editing, while adjustments to the relative positions of its constituent blocks should occur within its composition boundary. This hierarchical distinction prevents external layout changes from inadvertently disrupting the composition's internal structure.

4.2 Compatibility and Alternative Recommendation

Expressing a composition intent does not guarantee that the participating charts can be meaningfully combined. Charts may differ in data dimensionality, field types, levels of granularity, or coordinate systems. Moreover, compatibility depends on the intended composition: charts that are unsuitable for layering may still be valid when concatenated, whereas sharing an axis or forming a facet introduces additional data constraints. Compatibility should therefore be evaluated in the context of both the chart blocks and their intended relationship.

Evaluate compatibility in the context of composition. Compatibility is not an intrinsic property of two chart blocks. Each composition operation introduces different requirements for their data, encodings, and coordinate systems. Compatibility assessment should therefore consider the intended operation and identify which properties support or prevent the proposed composition.

Turn incompatibilities into actionable alternatives. Simply rejecting an incompatible composition provides little guidance for continued authoring. Instead, incompatibilities should serve as opportunities to suggest chart alternatives or data mappings that make the intended composition feasible. Such recommendations can help users resolve data mismatches without reconstructing the constituent charts.

Organize alternatives along meaningful design dimensions. Our formative study identified coordinate system, visual style, and data dimensionality as three dimensions for characterizing chart blocks and their alternatives. These dimensions provide a structured basis for recommending alternatives that vary selected properties while preserving others. For example, an alternative may retain the data and analytical role of a chart while changing its visual style or coordinate system to better support the intended composition.

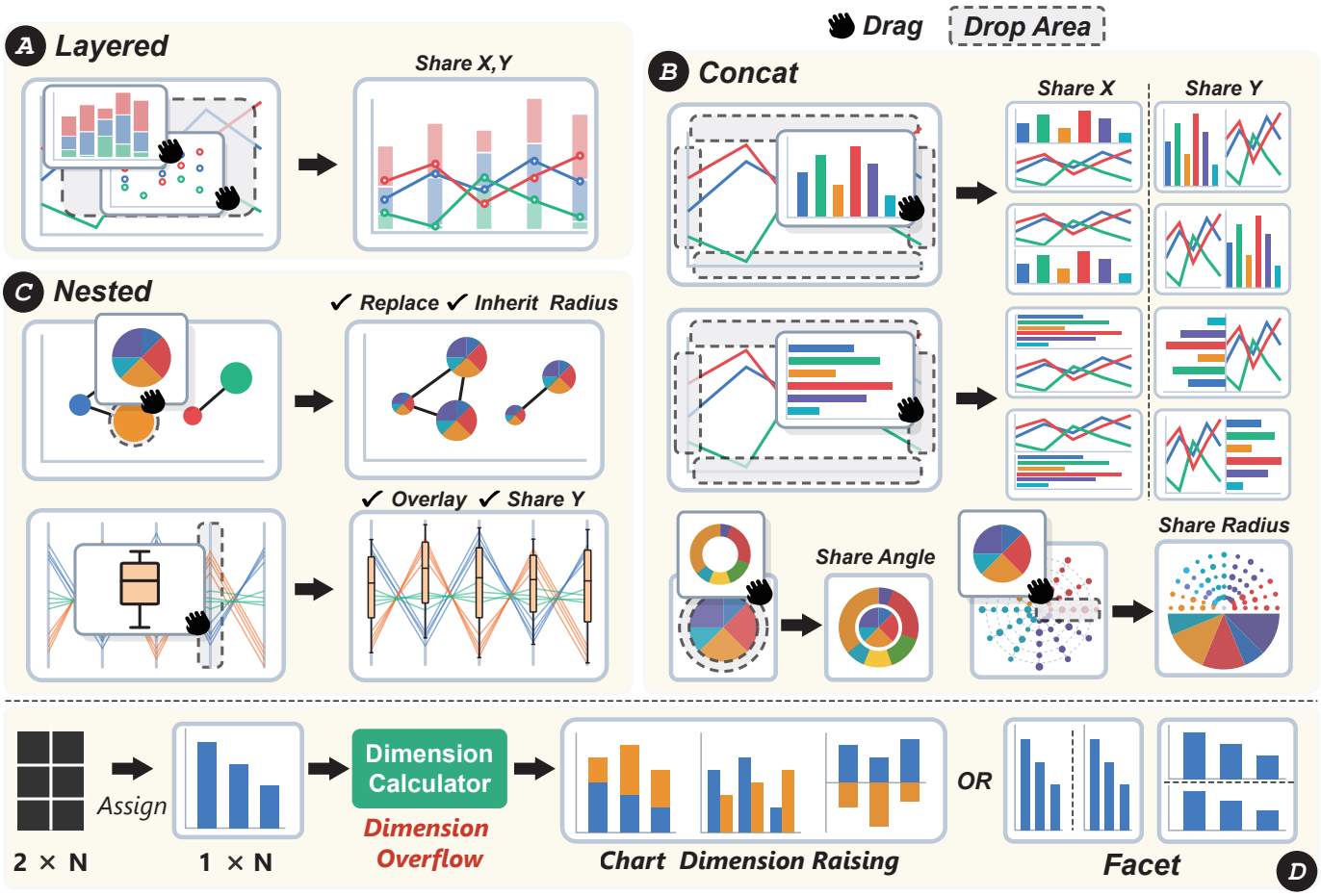

Figure 3: Drag-and-drop interactions for composing chart blocks. (A) Dropping a block into another block's plotting region creates layering by sharing both spatial dimensions. (B) Dropping along a boundary creates concatenation; the boundary determines the shared dimension and block order. The same interaction extends to polar coordinates through angular and radial boundaries. (C) Dropping onto an internal mark or structural element creates nesting, replacing or overlaying repeated elements while inheriting their data and spatial references.

5 CompoVisor

5.1 Interaction of Composition

CompoVisor supports composition through both explicit controls and drag-and-drop, with drag-and-drop serving as the primary interaction for direct manipulation (Fig. 3). Authors can invoke a composition operation using dedicated controls or express the same intent spatially by dragging one block onto a corresponding drop area of another. During dragging, valid target regions are visualized as drop areas. Each drop area indicates not only whether the two blocks can be composed, but also the resulting composition type, shared spatial dimensions, and block ordering. Dropping outside these regions leaves the blocks independently positioned in the free-form layout.

Identifying composition targets. When an author drags one block over another, CompoVisor highlights valid drop areas based on their data and coordinate compatibility. Moving across these areas previews the available composition operations before committing a drop.

Layering through an interior drop. Dropping a block into the plotting region of another block creates a layered composition (Fig. 3.A). The two blocks are placed in a common coordinate space and share both spatial dimensions. For example, dropping a line chart into the plotting region of a bar chart aligns their x and y encodings and produces a combined bar-and-line visualization. The plotting region acts as a two-dimensional drop target, distinguishing layering from blocks that merely overlap in the free-form layout.

Concatenation through boundary drops. Dropping a block along the boundary of another block creates a concatenated composition (Fig. 3.B). The selected boundary determines both the shared dimension and the relative order of the blocks. A drop above or below the target produces vertical concatenation and shares the x dimension, whereas a drop to its left or right produces horizontal concatenation and shares the y dimension. Opposing boundary regions allow authors to determine which block appears first without invoking a separate reordering command. The same interaction is interpreted according to the coordinate system of the participating blocks. In polar coordinates, a radial boundary drop creates concentric regions that share the angular dimension, while an angular boundary drop creates adjacent sectors that share the radial dimension. Authors thus use the same boundary-based gesture across coordinate systems, while the resulting spatial relationship follows the dimensions of the target coordinate space.

Nesting through element-level drops. The interaction proceeds from nesting to positioning and composition. First, the author drops a child block onto an eligible mark or structural element (Fig. 3.C), prompting CompoVisor to associate the child with the corresponding class of parent elements, instantiate it for each eligible element, and establish the required data correspondence. The child is initially aligned to the center of its parent. If further adjustment is needed, the author can open the positioning interface (Fig. 4) and independently select an anchor on the parent and the child; then use either quick-alignment presets or drag the child from this anchor-aligned position, with the displacement recorded as an (x, y) offset.

Editing composed blocks. After a drop is committed, the resulting composition forms a new manipulation boundary. In the surrounding layout, it behaves as a coherent unit: moving or resizing the composition preserves the relative positions and shared spatial relationships of its constituent blocks. To modify these internal relationships, such

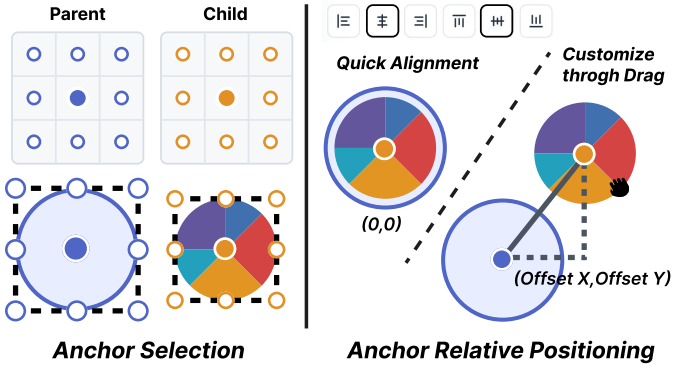

Figure 4: Anchor-based relative positioning between a parent and child. The selected parent and child anchors define the initial alignment, which can be further refined through direct dragging and an (x, y) offset.

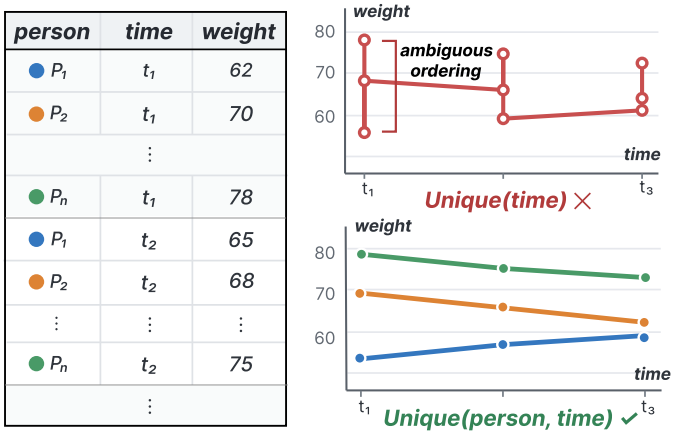

Figure 5: A single-line chart requires *time* to be unique, which is violated when multiple people share the same time value. A multi-line chart instead requires each (*person*, *time*) pair to be unique, allowing the observations to form separate trajectories..

as changing the order of concatenated blocks or adjusting the placement of a nested child, the author enters the composition and edits its constituent blocks directly. This distinction prevents external layout operations from inadvertently disrupting an established composition while retaining access to its internal structure.

5.2 Compatibility Engine

The compatibility engine determines whether a dataset can support a candidate chart type under a given channel assignment. Rather than reasoning about rendering properties such as marks, axes, or coordinate systems, the engine focuses exclusively on the data requirements of a chart. These requirements are represented by a *data contract*, and contract violations are resolved, when possible, through a *minimal fix*.

5.2.1 Data Contract

We describe the data requirements of each chart block using a data contract. A data contract specifies the required visual channels, the types of fields that can be assigned to them, and the fields that jointly form the chart key. An assignment satisfies the contract when each field is compatible with its channel and the key uniquely identifies the observations required by the chart. We refer to these two conditions as *type compatibility* and *key uniqueness*, respectively.

Type compatibility. Consider a single-line chart whose observations are encoded by horizontal position x and vertical position y. Because the x-channel determines both horizontal position and connection order, it accepts an ordered, temporal, or continuous quantitative field. The y-channel determines vertical position and therefore requires a quantitative field. Assigning a nominal field to y, for example, violates type compatibility because it does not provide the numerical values needed to position observations vertically. A multiline chart additionally contains a *series* channel, which partitions observations into separate lines and therefore requires a discrete field.

Key uniqueness. Type compatibility alone does not ensure that the assigned data form a valid chart. In Figure 5, assigning *time* to the x-channel and *weight* to the y-channel satisfies the type requirements of a single-line chart. However, the data contain one observation for each person at every time value. Without *person*, these observations are interpreted as points on the same line. Ordering and connecting them by *time* introduces vertical segments whose connection order has no meaningful interpretation. A single-line chart therefore uses x as its key and requires each x value to identify at most one observation:

$$\text{Unique}_D(x).$$

A multiline chart accommodates the same data by assigning *person* to the series channel. This assignment partitions the observations into separate trajectories and adds the series field to the key:

$$\text{Unique}_D(\textit{series}, x).$$

Neither x nor series needs to be unique; their combination must identify each observation. Thus, observations may share an x value across different series, but not within the same series unless they are first aggregated. More generally, a chart key specifies the assigned fields that must jointly distinguish its observations. Unlike type compatibility, which can be checked for each channel independently, key uniqueness must be evaluated jointly over these fields.

Contract inheritance in composition. Given chart blocks that satisfy their local contracts, composition introduces additional constraints on how their data and encodings relate across blocks. *Layering* places blocks in a shared coordinate system and therefore requires compatibility along both spatial dimensions. Corresponding channels must have compatible data types and semantics; discrete channels must have consistent or reconcilable domains, whereas quantitative and temporal channels must use comparable scales. *Concatenation* requires compatibility only along the shared coordinate dimension. Channels on this dimension must have compatible data types, semantics, domains, and scales, while channels on the partitioned dimension remain independent. *Faceting* partitions a block by a discrete field into a finite set of chart instances. The same contract is applied to each partition so that all instances maintain a consistent visual structure. *Nesting* embeds a child block within elements of a parent block. Each child instance can obtain its input data from the corresponding parent element, either directly from its bound record or through a shared key that retrieves related records. The resulting data must provide the fields required by the child contract. A composition is accepted only when these operator-specific constraints are satisfied. Otherwise, the composition is disabled until the participating blocks are configured compatibly.

5.2.2 Contract Validation and Repair

Our system validates channel, relational, and composition contracts at different stages of the authoring workflow. These contracts are used not only to detect invalid charts, but also to prevent invalid operations and guide users toward valid alternatives. Channel and composition violations are mainly prevented through interactive constraints, whereas relational violations are resolved through explicit repair.

Channel contracts are applied while the user edits a chart block in the encoding configuration panel. For each visual channel, the contract specifies its required field type, cardinality, and other role-specific constraints. The system evaluates the available columns against these constraints and disables incompatible choices. Invalid channel bindings are therefore prevented at selection time, rather than detected and repaired after the chart has been constructed.

Relational contracts are validated when the user confirms the chart configuration. The system checks whether the selected visual dimensions functionally determine the encoded values when such a depen-

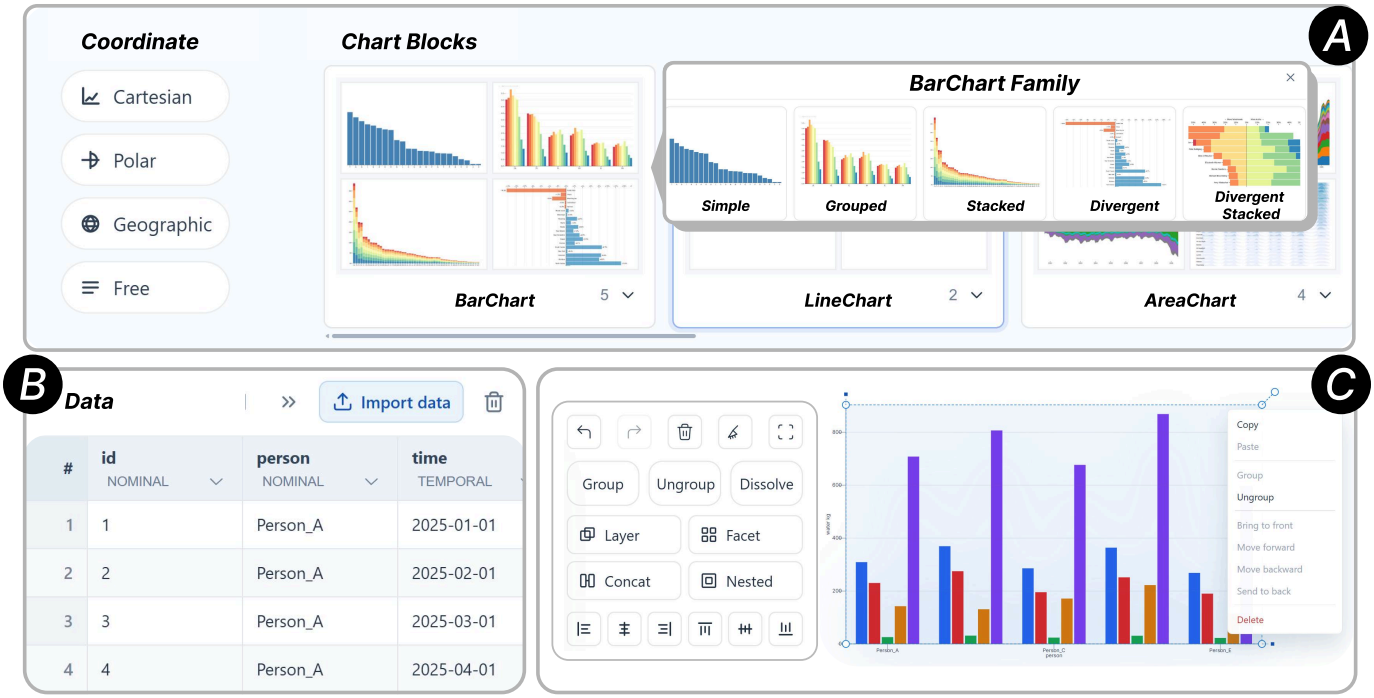

Figure 6: interface.

dency is required by the chart contract. If this condition is violated, the system identifies all inclusion-minimal sets of columns that can distinguish the conflicting records. A set is inclusion-minimal if none of its proper subsets can resolve the violation; sets of different sizes are retained as long as they are independently minimal. The user first selects one candidate set and then resolves its columns incrementally. For each column, the system offers three repair strategies: aggregating the conflicting values with an operation such as sum or mean; introducing the column as an additional visual dimension, which may upgrade the chart from, for example, a bar chart to a stacked or grouped bar chart; or using the column to facet the chart. After each repair operation, the relational contract is revalidated until all conflicts are resolved.

Composition contracts are used both before and after chart blocks are composed. Before composition, the system validates the local contracts of the participating blocks and checks their compatibility according to the selected composition operator. For example, layering requires compatibility along both dimensions of the shared coordinate system, whereas concatenation requires compatibility only along the shared dimension. An incompatible composition is disabled rather than repaired automatically. Once a composition has been created, its compatibility constraints are propagated back to the encoding configuration panels of its constituent blocks. Consequently, editing one block is constrained by the contracts of the other blocks, and columns that would invalidate the existing composition become unavailable.

5.3 Interface

The system is organized as a canvas-based authoring workspace consisting of a *Block List*, a *Data Panel*, and an *Encoding Config Panel*.

Canvas (Figure 6.C) is the main workspace for arranging and composing chart blocks. It supports basic editing and layout operations on a zoomable and pannable board. Chart compositions with drag-and-drop are also performed directly on the Canvas.

Block List (Figure 6.A) is located at the top of the sidebar and provides access to available chart templates. To reduce the search space, templates are first filtered by coordinate system: Cartesian, Polar, Geographic, or coordinate-free. Within each coordinate system, they are grouped into families such as bar, line, and area. Each family is shown as a compact preview with representative thumbnails and a template count. Selecting a family reveals its templates, which can then be dragged onto the canvas and instantiated as chart blocks.

Data Panel (Figure 6.B) supports importing and inspecting CSV data. Tabular data is imported as a single CSV file, while graph data is imported as two CSV files, one for nodes and one for edges. After import, users can preview the data and adjust the inferred field types to provide appropriate semantics. These types are then used to check whether fields are compatible with the visual roles of a chart block.

Encoding Config Panel provides configuration options for the selected chart block. Using a simplified Vega-Lite-style specification, it allows users to bind imported data to the block and map data fields to its visual channels. The available options are determined by the selected chart template and its encoding requirements. The panel also checks the compatibility between the data and the chart. When an incompatibility is detected, it displays the issue and provides applicable resolution options, such as data aggregation, chart upgrade, or faceting. The configuration can be confirmed only after all incompatibilities have been resolved.

Together, these components support an iterative workflow: authors import and inspect data, browse and place a chart block, configure its encodings, arrange and compose charts on the Canvas, and resolve any remaining structural incompatibilities through explicit repair choices.

6 User Study

We conducted a controlled user study to evaluate whether users could understand and apply our framework to author composite visualizations. We adopted visualization re-creation tasks because they provide consistent targets, cover the core authoring operations, and enable a controlled comparison between systems. We compared our system with Lyra using two VA designs selected from the VAID corpus.

6.1 Study Setting

We next describe the participants, tasks, and procedure of the study.

Tasks. We selected two VA designs of comparable authoring complexity from the VAID corpus as re-creation tasks. Based on a decomposition agreed upon by visualization experts, Task 1 contained [X] chart blocks and approximately [M] composites, whereas Task 2 contained [Y] chart blocks and approximately [N] composites. The composite counts are approximate because the same target design may be reproduced using different composition strategies. Participants were asked

to reproduce the main charts and their composition while ignoring annotations and minor stylistic details.

Participants. We recruited 12 participants (P1–P12) with experience in visualization authoring, including dashboard creation, VA system design, and visualization research or development. They were familiar with visualization tools or grammars such as D3, Vega, or Vega-Lite. Participants were evenly divided into two groups of six. One group completed Task 1 with CompoVisor and Task 2 with Lyra, while the other used Lyra for Task 1 and CompoVisor for Task 2, thereby counterbalancing the task–system assignment and system order.

Procedure. Each session lasted approximately two hours. For each system, participants received a 15-minute tutorial using a practice example distinct from the study tasks, followed by 5 minutes of free exploration and a 25-minute re-creation task. Before each task, we introduced the input data and explained the visual structure of the target design. When participants were blocked, the facilitator clarified system operations without making authoring decisions on their behalf. During each task, we recorded task completion, completion time, and the number of mouse clicks. After each task, participants completed a 7-point Likert-scale questionnaire assessing ease of learning, ease of use, authoring efficiency, support for chart composition, and overall satisfaction. Finally, we conducted a 15-minute semi-structured interview organized around three topics: (1) how the block granularity of CompoVisor, compared with that of Lyra, affected authoring; (2) how the composite construction processes of the two systems differed; and (3) the reasons behind participants' questionnaire ratings. We also collected feedback on their authoring strategies, encountered difficulties, and suggested improvements.

6.2 Quantitative Results

Figure 3 summarizes the questionnaire ratings and task performance under the two conditions. We report questionnaire ratings, completion time, and mouse clicks using medians and interquartile ranges because of the limited sample size. We used Wilcoxon signed-rank tests for paired comparisons of questionnaire ratings, completion time, and mouse clicks, and an exact McNemar test to compare task completion rates. We report effect sizes together with the statistical significance of the differences.

Subjective ratings. Participants rated CompoVisor higher than Lyra in [dimensions], with median ratings of [X] and [Y], respectively. The differences in [dimensions] were statistically significant ($p = [\,]$, $r = [\,]$), whereas no significant difference was observed for [dimensions]. These results suggest that [interpretation based on the actual results].

Task performance. Participants successfully completed [X/12] tasks with CompoVisor and [Y/12] with Lyra. Among the completed tasks, the median completion times were [X] and [Y] minutes, and the median numbers of mouse clicks were [X] and [Y], respectively. CompoVisor resulted in [shorter/similar] completion times and [fewer/a similar number of] interactions than Lyra. This difference indicates that [interpretation], although [mention nonsignificance or task variation if applicable].

6.3 Qualitative Results

We analyzed participants' interview responses and organized the findings around three themes: block-level authoring and family organization, composite construction workflows, and the rationale underlying participants' subjective ratings.

Block-level authoring and family organization. Participants perceived different trade-offs between the block granularities of CompoVisor and Lyra. [X] participants considered the blocks in CompoVisor [more suitable/easier to manipulate] because [reason and representative behavior]. In contrast, [X] participants preferred Lyra's [finer/coarser] granularity when [scenario]. Participants also reported that the family-based design helped them [identify related blocks, reuse configurations, or manage structural consistency]. However, [limitations or learning difficulties, if any]. These observations indicate that [overall interpretation].

Composite construction workflow. Participants described the two systems as supporting different composition strategies. With CompoVisor, they typically [describe the actual workflow], whereas with Lyra, they [describe the corresponding Lyra workflow]. Participants noted that CompoVisor's composite operations made [relationships/layout structures] more explicit and reduced [specific operations or difficulties]. By comparison, Lyra offered [its perceived advantage], but required participants to [specific additional operation]. Several participants also noted that [common difficulty or limitation]. Overall, these findings suggest that [interpretation of the workflow difference].

Perceived usability and rating rationale. Participants attributed their ratings primarily to [factors]. Those who rated CompoVisor highly emphasized [advantages], while lower ratings were mainly associated with [limitations]. Ratings for Lyra were influenced by [factors]. These explanations help contextualize the quantitative results: [connect the interview findings to the differences or lack of differences in questionnaire ratings].

7 Discussion

8 Conclusion

References

[1] Tableau. https://www.tableau.com/. 1, 2

[2] M. Bostock and J. Heer. Protovis: A graphical toolkit for visualization. *IEEE Transactions on Visualization and Computer Graphics*, 15(6):1121–1128, 2009. doi: 10.1109/TVCG.2009.174 2

[3] M. Bostock, V. Ogievetsky, and J. Heer. D^3 data-driven documents. *IEEE Transactions on Visualization and Computer Graphics*, 17(12):2301–2309, 2011. doi: 10.1109/TVCG.2011.185 1, 2, 3

[4] R. Chen, X. Shu, J. Chen, D. Weng, J. Tang, S. Fu et al. Nebula: A coordinating grammar of graphics. *IEEE Transactions on Visualization and Computer Graphics*, 28(12):4127–4140, 2022. doi: 10.1109/TVCG.2021.3076222 2

[5] Datawrapper. Datawrapper. https://www.datawrapper.de/, 2026. Accessed: 2026-08-05. 2

[6] D. Deng, W. Cui, X. Meng, M. Xu, Y. Liao, H. Zhang et al. Revisiting the design patterns of composite visualizations. *IEEE Transactions on Visualization and Computer Graphics*, 29(12):5406–5421, 2023. doi: 10.1109/TVCG.2022.3213565 1, 2

[7] Z. Deng, D. Weng, S. Liu, Y. Tian, M. Xu, and Y. Wu. A survey of urban visual analytics: Advances and future directions. *Computational Visual Media*, 9(1):3–39, 2023. doi: 10.1007/S41095-022-0275-7 1, 2

[8] W. Javed and N. Elmqvist. Exploring the design space of composite visualization. In *Proceedings of IEEE Pacific Visualization Symposium*, pp. 1–8, 2012. 1, 2

[9] J. Jo, F. Vernier, P. Dragicevic, and J. Fekete. A declarative rendering model for multiclass density maps. *IEEE Transactions on Visualization and Computer Graphics*, 25(1):470–480, 2019. doi: 10.1109/TVCG.2018.2865141 2

[10] X. Kui, N. Liu, Q. Liu, J. Liu, X. Zeng, and C. Zhang. A survey of visual analytics techniques for online education. *Visual Informatics*, 6(4):67–77, 2022. 1, 2

[11] Z. Liu, C. Chen, F. Morales, and Y. Zhao. Atlas: Grammar-based procedural generation of data visualizations. In *2021 IEEE Visualization Conference, IEEE VIS 2021 - Short Papers, New Orleans, LA, USA, October 24-29, 2021*, pp. 171–175. IEEE, 2021. 2

[12] Z. Liu, J. Thompson, A. Wilson, M. Dontcheva, J. Delorey, S. Grigg et al. Data illustrator: Augmenting vector design tools with lazy data binding for expressive visualization authoring. In *Proceedings of the 2018 CHI Conference on Human Factors in Computing Systems*, CHI '18, pp. 1–13. Association for Computing Machinery, New York, NY, USA, 2018. doi: 10.1145/3173574.3173697 1, 2

[13] S. LYi, J. Jo, and J. Seo. Comparative layouts revisited: Design space, guidelines, and future directions. *IEEE Transactions on Visualization and Computer Graphics*, 27(2):1525–1535, 2021. doi: 10.1109/TVCG.2020.3030419 2

[14] S. L'Yi, Q. Wang, F. Lekschas, and N. Gehlenborg. Gosling: A grammar-based toolkit for scalable and interactive genomics data visualization. *IEEE Transactions on Visualization and Computer Graphics*, 28(1):140–150, 2022. doi: 10.1109/TVCG.2021.3114876 2

[15] M. Mauri, T. Elli, G. Caviglia, G. Uboldi, and M. Azzi. RAWGraphs: A Visualisation Platform to Create Open Outputs. In *Proceedings of the 12th Biannual Conference on Italian SIGCHI Chapter*, pp. 1–5. ACM, 2017. doi: 10.1145/3125571.3125585 2

[16] G. Moreira, M. Hosseini, M. N. A. Nipu, M. Lage, N. Ferreira, and F. Miranda. The urban toolkit: A grammar-based framework for urban visual analytics. *IEEE Transactions on Visualization and Computer Graphics*, 30(1):1402–1412, 2024. doi: 10.1109/TVCG.2023.3326598 2

[17] T. Munzner. A nested model for visualization design and validation. *IEEE transactions on visualization and computer graphics*, 15(6):921–928, 2009. 2

[18] D. Park, S. M. Drucker, R. Fernandez, and N. Elmqvist. Atom: A grammar for unit visualizations. *IEEE Transactions on Visualization and Computer Graphics*, 24(12):3032–3043, 2018. doi: 10.1109/TVCG.2017.2785807 2

[19] Plotly Technologies Inc. Plotly. `https://plotly.com/`, 2026. Accessed: 2026-08-05. 2

[20] M. Rodrigues, J. Dapello, P. Vaithilingam, C. Nobre, and J. Beyer. Protograph: A non-expert toolkit for creating animated graphs. In *2023 IEEE Visualization and Visual Analytics (VIS), Melbourne, Australia, October 21-27, 2023*, pp. 176–180. IEEE, 2023. 2

[21] A. Satyanarayan and J. Heer. Lyra: an interactive visualization design environment. In *Proceedings of the 16th Eurographics Conference on Visualization*, EuroVis '14, pp. 351–360. Eurographics Association, Goslar, DEU, 2014. 1, 2

[22] A. Satyanarayan, D. Moritz, K. Wongsuphasawat, and J. Heer. Vega-lite: A grammar of interactive graphics. *IEEE Transactions on Visualization and Computer Graphics*, 23(1):341–350, 2017. doi: 10.1109/TVCG.2016.2599030 1, 2, 3

[23] A. Satyanarayan, R. Russell, J. Hoffswell, and J. Heer. Reactive vega: A streaming dataflow architecture for declarative interactive visualization. *IEEE Transactions on Visualization and Computer Graphics*, 22(1):659–668, 2016. doi: 10.1109/TVCG.2015.2467091 2

[24] A. Satyanarayan, K. Wongsuphasawat, and J. Heer. Declarative interaction design for data visualization. In H. Benko, M. Dontcheva, and D. Wigdor, eds., *The 27th Annual ACM Symposium on User Interface Software and Technology, UIST '14, Honolulu, HI, USA, October 5-8, 2014*, pp. 669–678. ACM, 2014. doi: 10.1145/2642918.2647360 2

[25] M. Shih, C. Rozhon, and K. Ma. A declarative grammar of flexible volume visualization pipelines. *IEEE Transactions on Visualization and Computer Graphics*, 25(1):1050–1059, 2019. doi: 10.1109/TVCG.2018.2864841 2

[26] F. B. Viegas, M. Wattenberg, F. van Ham, J. Kriss, and M. McKeon. Many Eyes: A Site for Visualization at Internet Scale. *IEEE Transactions on Visualization and Computer Graphics*, 13(6):1121–1128, 2007. doi: 10.1109/TVCG.2007.70577 2

[27] K. Wongsuphasawat. Encodable: Configurable grammar for visualization components. In *31st IEEE Visualization Conference, IEEE VIS 2020 - Short Papers, Virtual Event, USA, October 25-30, 2020*, pp. 131–135. IEEE, 2020. 2

[28] A. Wu, Y. Wang, X. Shu, D. Moritz, W. Cui, H. Zhang et al. Ai4vis: Survey on artificial intelligence approaches for data visualization. *IEEE Transactions on Visualization and Computer Graphics*, 28(12):5049–5070, 2022. doi: 10.1109/TVCG.2021.3099002 1, 2

[29] Y. Wu, N. Cao, D. Gotz, Y. Tan, and D. A. Keim. A survey on visual analytics of social media data. *IEEE Transactions on Multimedia*, 18(11):2135–2148, 2016. doi: 10.1109/TMM.2016.2614220 1, 2

[30] H. Xia, N. Henry Riche, F. Chevalier, B. De Araujo, and D. Wigdor. Dataink: Direct and creative data-oriented drawing. In *Proceedings of the 2018 CHI Conference on Human Factors in Computing Systems*, CHI '18, pp. 1–13. Association for Computing Machinery, New York, NY, USA, 2018. doi: 10.1145/3173574.3173797 2

[31] L. Ying, A. Wu, H. Li, Z. Deng, J. Lan, J. Wu et al. VAID: indexing view designs in visual analytics system. In F. F. Mueller, P. Kyburz, J. R. Williamson, C. Sas, M. L. Wilson, P. O. T. Dugas et al., eds., *Proceedings of the CHI Conference on Human Factors in Computing Systems, CHI 2024, Honolulu, HI, USA, May 11-16, 2024*, pp. 198:1–198:15. ACM, 2024. doi: 10.1145/3613904.3642237 1, 2, 3

[32] Y. Zhao, Y. Wang, X. Luo, Y. Wang, and J.-D. Fekete. Libra: An interaction model for data visualization. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*, CHI '25, art. no. 1169, 17 pp. Association for Computing Machinery, New York, NY, USA, 2025. doi: 10.1145/3706598.3713769 2

[33] T. Zhou, J. Huang, and G. Y.-Y. Chan. Epigraphics: Message-driven infographics authoring. In *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems*, CHI '24, art. no. 200, 18 pp. Association for Computing Machinery, New York, NY, USA, 2024. doi: 10.1145/3613904.3642172 2

[34] J. Zong, J. Pollock, D. Wootton, and A. Satyanarayan. Animated vega-lite: Unifying animation with a grammar of interactive graphics. *IEEE Transactions on Visualization and Computer Graphics*, 29(1):149–159, 2023. doi: 10.1109/TVCG.2022.3209369 2